

Proof. By symmetry, $\mathbb{E}[\sin(bx)] = 0$. We calculate

$$\text{Cov}(x, \sin(bx)) = (2\pi)^{-1/2} \int_{-\infty}^{\infty} x \sin(bx) e^{-x^2/2} dx = b e^{-b^2/2}.$$

We already computed the variance in Lemma 14, so

$$\rho(x, \sin(bx)) = \frac{\text{Cov}(x, \sin(bx))}{\sqrt{\text{Var}(x)\text{Var}(\sin(bx))}} = \frac{2be^{3b^2/2}}{e^{2b^2} - 1}.$$

In our experiments we set $b = 10$, so $\rho(x, \sin(bx)) < 10^{-20}$.

D Details and additional experiments

D.1 Synthetic setting: $\mathcal{D}_p$ task

We use a smaller version of the GPT-2 architecture, adapted to regression tasks. That is, the token embedding and unembedding layers are replaced with a trained linear map from the input space to the embedding space and from the embedding space to the output space, respectively. For each $p \in \{0.01, 0.1, 0.3, 1\}$ models are trained using ordinary and myopic descent on one epoch of 30M sequences of length 64 sampled from $\mathcal{D}_{p,10,10}^{64}$. See Table 3 for architecture details.

Configuration Key	Value
num_layers	2
num_heads	2
embd_dim	128
n_inner	512
input_dim	2
output_dim	1
activation	relu
attn_pdrop	0
embd_pdrop	0
resid_pdrop	0
lr	1e-3
optimizer	AdamW
weight_decay	0.01
betas	(0.9, 0.999)
scheduler	cosine
warmup	0.01
batch_size	512
seq_length	64
loss_fn	HuberLoss

Table 3: Transformer configuration used when training on synthetic data distribution $\mathcal{D}_p$

D.1.1 Probe details

Given a transformer model trained on $\mathcal{D}_p$, we sample the hidden states at each layer when the model is given as input 50 000 evaluation sequences from the same distribution $\mathcal{D}_p$. For each layer and targets $\sin(bx_{n-i})$ for varying $i > 0$, we fit a linear regression model on the hidden state of that layer to the target. The in-sample R^2 of each linear model is then reported in Figure 2. Figure 6 is a visualization of the linear probe’s performance on the vanilla transformer.

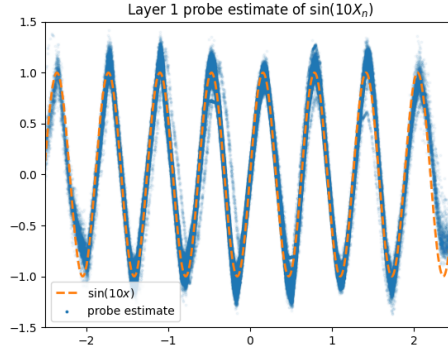


Figure 6: Estimate of $\sin(bx_n)$ by linear probe fit on layer 1 of transformer with vanilla training on $\mathcal{D}_{0.3}$. Computed over 50 000 samples from $\mathcal{D}_1$.

D.2 Natural language setting

D.2.1 GPT-2

For both vanilla and myopic training, we train the GPT2-small architecture from random initialization for one epoch on 4.6M sequences from the MS MARCO dataset (Nguyen et al., 2016), truncated to length 64. To estimate the local myopia bonus of the vanilla model, we train another model from random initialization with the same architecture, but with past hidden states provided by the vanilla model. Note that this “local myopic” model attains slightly *better* performance than the vanilla model; each forward pass can focus purely on next-token prediction, since past hidden states are supplied by a separate model. As a baseline, we also train a “transformer bigram” model, which is a transformer model whose key/value states are zeroed out during training and evaluation. See Table 4 for configuration details.

Configuration Key	Value
num_layers	12
num_heads	12
embd_dim	768
n_inner	3072
vocab_size	50257
activation	gelu_new
attn_pdrop	0.1
embd_pdrop	0.1
resid_pdrop	0.1
lr	6.0×10^{-4}
optimizer	AdamW
weight_decay	0.01
betas	(0.9, 0.999)
scheduler	cosine
warmup	0.01
batch_size	512
seq_length	64
loss_fn	CrossEntropy

Table 4: GPT-2 small configuration used when training on natural language data.

D.2.2 Pythia

For experiments on the Pythia suite, we finetune with either vanilla or myopic descent on 10M sequences of 64 tokens each subsampled from The Pile (Gao et al., 2020). Learning rates and batch sizes for each model are presented in Table 5; they are the same between vanilla and myopic descent. All other training and architectural hyperparameters are the same as those used by Biderman et al. (2023).

Model	Learning rate	Batch size
Pythia 14M	4.0×10^{-4}	512
Pythia 31M	4.0×10^{-4}	512
Pythia 70M	1.2×10^{-4}	512
Pythia 160M	1.2×10^{-4}	512
Pythia 410M	1.2×10^{-4}	256
Pythia 1B	1.2×10^{-4}	128
Pythia 1.4B	1.2×10^{-4}	128
Pythia 2.8B	8.0×10^{-5}	64

Table 5: Pythia suite hyperparameters for finetuning. Batch size is measured in sequences of 64 tokens each.

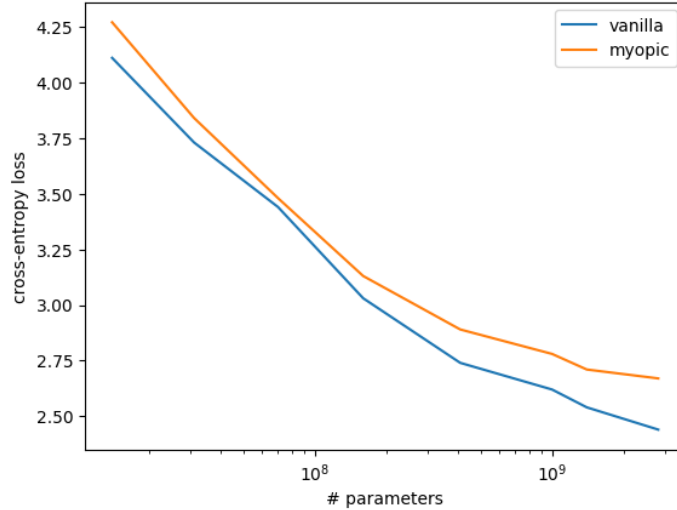


Figure 7: Cross-entropy loss of Pythia models fine-tuned on The Pile dataset using vanilla and myopic gradient descent. Starting from the 70M model, we see that the gap increases with parameter count.

D.3 Multiplication

In addition to the natural language experiments, we also measure the myopia gap on the task of natural number multiplication. We use the same GPT2-small architecture as in the natural language experiments starting from random initialization; see Table 4. See Figure 8 for an example input sequence.

$$3700 * 5400 = 58230000$$

Figure 8: Example multiplication input sequence.